Stock Price Prediction Using Recurrent Neural Networks: A Comparative Study of RNN, LSTM, and GRU Models

Rubaiyat Alam Ruhin
The University of Adelaide
Adelaide SA 5005
a1930497@adelaide.edu.au

Abstract

In this study, I explored Recurrent Neural Networks (RNNs), including Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) architectures, for predicting Google stock Adjusted Close prices. Using historical data spanning nearly two decades (2004–2023), I compared the performance of these models. The GRU model, optimized with hyperparameters, achieved a Mean Squared Error (MSE) of 0.00177 and a Root Mean Squared Error (RMSE) of 0.04203 on the test dataset. It also captured overall price trends with a Mean Directional Accuracy (MDA) of 48.61%, outperforming the baseline Simple RNN and LSTM models. However, the GRU's MDA highlighted challenges in predicting directional changes. Hyperparameter optimization reduced the validation loss to 0.000147, enhancing model performance and generalization. These findings demonstrate the potential of deep learning models for stock price forecasting and provide insights into selecting appropriate architectures. Future work may integrate additional features and explore ensemble models for improved predictive accuracy.

1. Introduction

Stock price prediction is a challenging problem in the domain of financial time series forecasting due to the inherent complexity and volatility of stock market data. Accurate predictions can provide significant benefits, ranging from assisting investors in decision-making to enabling algorithmic trading systems. However, traditional statistical methods often struggle to capture the non-linear patterns and sequential dependencies present in financial data.

In this work, I investigated the use of Recurrent Neural Networks (RNNs) and their advanced variants, Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU), to predict the Adjusted Close price of Google stocks using historical data spanning from 2004 to 2023. These models are particularly well-suited for sequential data due to their ability to learn temporal dependencies. By leveraging deep learning frameworks, I aimed to address the limitations of classical approaches and improve predictive accuracy.

The methodology involved preprocessing stock price data, normalizing it, and creating time-series sequences for model training and validation. Each architecture was implemented, trained, and evaluated using metrics like MSE, RMSE, and MDA. After a baseline comparison, the GRU model underwent hyperparameter optimization to enhance predictive capabilities.

This study compares RNN-based models for stock price prediction and demonstrates the impact of hyperparameter tuning on model performance and generalization. Insights highlight the potential of deep learning in financial forecasting and adaptability to complex real-world datasets.

2. Background

Stock price prediction has been challenging, with various methods proposed. Traditional statistical approaches like ARIMA models provide a foundation for time series forecasting but fail to capture nonlinear dependencies in stock market data, limiting accuracy in volatile scenarios.

Deep learning has advanced time series analysis, especially stock prediction. Recurrent Neural Networks (RNNs) excel at learning temporal dependencies and nonlinear relationships in sequential data. A study by Fischer and Krauss et al. [3] (2018) showed LSTMs outperform traditional machine learning models in stock return prediction. However, LSTMs face computational overhead and sensitivity to hyperparameters.

GRUs, introduced by Cho et al. [2] (2014), are computationally efficient alternatives to LSTMs, addressing some of their complexity while retaining comparable performance. Empirical analyses suggest GRUs converge faster than LSTMs, making them suitable for real-time applications.

Hybrid models, combining traditional and deep learn-

ing approaches, have gained traction. For instance, Patel et al. [5] (2015) integrated ARIMA models with neural networks, improving accuracy over standalone methods. However, these hybrid frameworks require significant computational resources and expertise.

Data preprocessing techniques enhance model performance. Studies like Goodfellow et al. [4] (2016) emphasize data normalization and augmentation for improved generalization and reduced overfitting. Inspired, I used Min-Max scaling to normalize stock data, accelerating convergence and enhancing predictions.

I compare Simple RNNs, LSTMs, and GRUs on Google stock price prediction. LSTMs have high learning capacity but high computational cost. GRUs balance accuracy and efficiency. Simple RNNs serve as a baseline. By tuning hyperparameters and optimizing data processing, I aim to understand these architectures in financial time series forecasting.

3. Methodology

3.1. Dataset

I have utilized the Google stock price dataset, retrieved via the yfinance Python library, to evaluate the predictive capabilities of Recurrent Neural Network (RNN) architectures. The dataset spans from January 1, 2004, to December 31, 2023, encompassing 4,875 data points. Each entry includes daily financial metrics: Open, High, Low, Close, Adjusted Close, and Volume. The Adjusted Close column reflects the closing price adjusted for corporate actions, making it relevant for time-series analysis. The Volume column shows the number of shares traded, indicating market activity. The dataset exhibits a mean adjusted close price of approximately 41.73 and a maximum value of 150.34, indicating significant price growth over the years. The trading volume ranges from 1.58 million to 1.65 billion, showcasing fluctuations in market engagement. This rich dataset serves as the foundation for exploring temporal dependencies and evaluating model performance in stock price forecasting tasks.

3.2. Exploratory Data Analysis

In the Exploratory Data Analysis (EDA), I examined the distributions and correlations of the stock price features. Figure 1 shows the distributions of key features, including `Adj Close`, `Close`, `High`, `Low`, `Open`, and `Volume`. Most features exhibit a right-skewed distribution, indicating a prevalence of lower values for stock prices, with a few higher values acting as outliers.

The correlation heatmap in Figure 2 revealed perfect correlation (1.00) among all price-related features (`Adj Close`, `Close`, `High`, `Low`, `Open`), making them redundant for predictive modeling. The `Volume` feature, however, showed a moderate negative correlation (-0.47) with the price-related features, suggesting that it may contribute distinct information to the model.

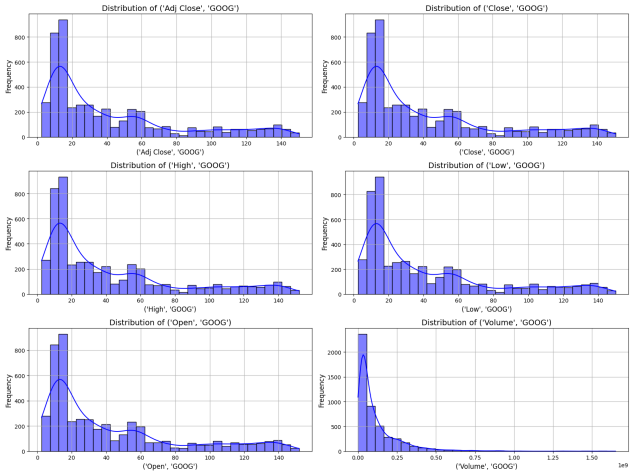


Figure 1. Distributions of stock price features. The features exhibit a right-skewed distribution, highlighting the prevalence of lower values.

This analysis guided the feature selection process, where redundant features were removed to improve the efficiency and focus of the predictive modeling process.

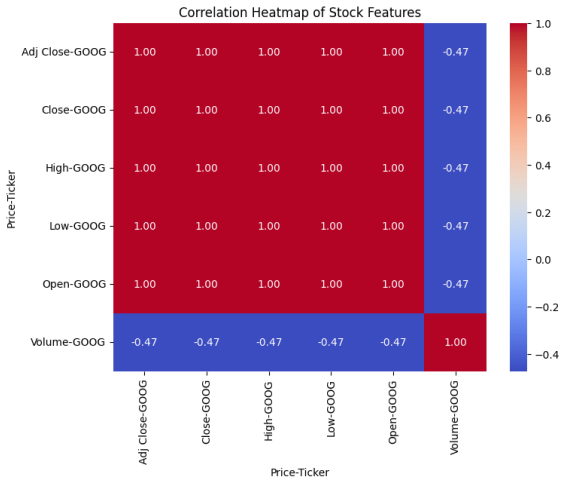


Figure 2. Correlation heatmap of stock price features. All price-related features are perfectly correlated (1.00), making them redundant for modeling purposes.

3.3. Preprocessing

During preprocessing, as shown in the workflow in Figure 3, I began by creating a new dataframe containing only two key features: `Adj Close` and `Volume`. This selection was informed by the redundancy observed in other features due to perfect correlations in the correlation heatmap.

Next, I applied Min-Max Scaling to normalize the selected features, scaling their values between 0 and 1 to facilitate the efficient training of Recurrent Neural Networks (RNNs). The normalized data was then split by year: data before 2020 was assigned to the training set, 2020 data was designated for validation, and data post-2020 was reserved for testing.

Finally, the data was prepared for RNN input by converting it into sequences. Each sequence comprised 30 time steps, with the target variable being the stock price (`Adj Close`) of the subsequent day.

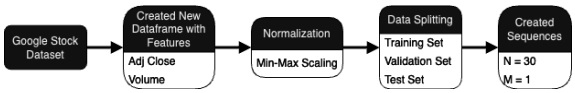


Figure 3. Workflow illustrating the preprocessing steps, including feature selection, normalization, data splitting, and sequence creation.

3.4. Deep Learning Models

3.4.1 Simple RNN

The Simple Recurrent Neural Network (RNN) is a foundational model for sequential data analysis. RNNs are designed to process sequences of inputs by maintaining a hidden state that captures dependencies between timesteps. This is particularly effective for time-series forecasting, such as predicting stock prices.

The model structure, as shown in Figure 4, involves a recurrent connection where the hidden state h_t at timestep t is computed using the previous hidden state h_{t-1} and the current input x_t. The computation is represented mathematically as:

$$h_t = \tanh(W \cdot h_{t-1} + U \cdot x_t + b_h)$$

where:

- W and U are weight matrices for the hidden state and input, respectively,
- b_h is the bias term,
- $\tanh$ is the activation function used for non-linearity.

The final output $\hat{y}_t$ at timestep t is computed as:

$$\hat{y}_t = V \cdot h_t + b_y$$

where V is the weight matrix for the output, and b_y is the bias term.

The model was trained using the Adam optimizer and a mean squared error loss function. The training process spanned 50 epochs with a batch size of 32.

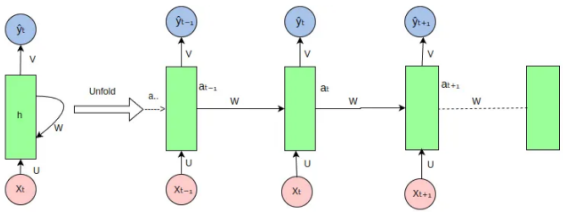


Figure 4. Architecture of the Simple RNN model. [6]

3.4.2 LSTM

Long Short-Term Memory (LSTM) networks are a type of Recurrent Neural Network (RNN) designed to address the vanishing gradient problem in standard RNNs. By introducing a memory cell, LSTMs allow selective retention and forgetting of information, enabling effective modeling of long-term dependencies in sequential data.

The architecture of the LSTM cell, depicted in Figure 5, includes three gates and a memory cell:

- **Forget Gate**: Determines which parts of the cell state to discard. This is calculated as:

  $$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

  where W_f is the weight matrix, h_{t-1} is the previous hidden state, x_t is the input, b_f is the bias, and σ is the sigmoid activation function.

- **Input Gate**: Selects which new information to update the cell state with:

  $$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

  $$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

  where i_t is the input gate, and $\tilde{C}_t$ represents candidate updates to the cell state.

- **Output Gate**: Determines the next hidden state based on the cell state:

  $$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

  $$h_t = o_t \cdot \tanh(C_t)$$

The cell state is updated using the forget and input gates:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

The model was compiled with the Adam optimizer and mean squared error loss function. Training was conducted over 50 epochs with a batch size of 32.

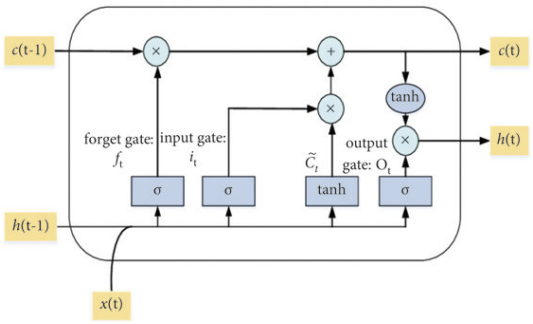


Figure 5. Illustration of the LSTM [7] cell architecture, highlighting the interactions of the forget gate, input gate, output gate, and cell state.

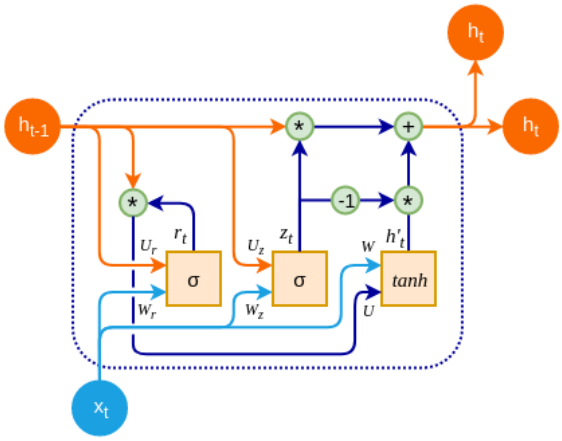


Figure 6. Structure of the GRU [1] cell, showing the interactions of the update gate, reset gate, candidate hidden state, and the final hidden state.

3.4.3 GRU

Gated Recurrent Units (GRUs) are a simplified variant of Long Short-Term Memory (LSTM) networks, designed to reduce computational complexity while maintaining the ability to model long-term dependencies in sequential data. Unlike LSTMs, GRUs have fewer parameters as they merge the forget and input gates into a single update gate, which makes them computationally efficient.

The GRU cell, depicted in Figure 6, consists of two primary gates:

- **Update Gate**: Determines the extent to which the previous hidden state (h_{t-1}) is carried forward and how much the current input (x_t) contributes to the new state:
$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$
- **Reset Gate**: Controls how much of the past information to forget:
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

The candidate hidden state ($\tilde{h}_t$) is computed as:

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t] + b)$$

The new hidden state (h_t) is a linear interpolation between the previous hidden state and the candidate hidden state:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

The GRU model was compiled using the Adam optimizer and mean squared error loss function, and trained over 50 epochs with a batch size of 32.

3.5. Hyperparameter Optimization

Hyperparameter optimization plays a crucial role in enhancing the performance of machine learning models by systematically searching for the best configuration of parameters. For this study, I implemented a two-step process for hyperparameter tuning: evaluating performance metrics for model comparison and conducting a grid search over a predefined parameter space.

3.5.1 Model Performance Evaluation

The performance of each model—Simple RNN, LSTM, and GRU—was evaluated using a weighted scoring system. I considered three key metrics:

- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE
- Mean Directional Accuracy (MDA)

Each metric was normalized to ensure comparability. A weighted score was computed using the formula:

$$\text{Weighted Score} = w_{\text{MSE}} \cdot \frac{1}{\text{MSE}} + w_{\text{RMSE}} \cdot \frac{1}{\text{RMSE}} + w_{\text{MDA}} \cdot \text{MDA}$$

where $w_{\text{MSE}} = 0.5$, $w_{\text{RMSE}} = 0.3$, and $w_{\text{MDA}} = 0.2$. The model with the highest score was selected for further optimization.

3.5.2 Hyperparameter Grid Search

I performed hyperparameter tuning on the GRU model by defining a comprehensive grid of hyperparameters:

- Number of units in the GRU layer: $\{32, 64, 128\}$.
- Number of layers: $\{1, 2\}$.
- Dropout rates: $\{0.1, 0.2, 0.3\}$.

- Batch sizes: $\{16, 32, 64\}$.
- Learning rates: $\{0.001, 0.01, 0.1\}$.
- Epochs: $\{20, 50, 100\}$.

To reduce computational overhead, I utilized random sampling to select 10 combinations from the hyperparameter space. For each combination, the GRU model was built and trained using the selected parameters. The final validation loss was recorded as the evaluation criterion for comparison.

The hyperparameter optimization process ensured a systematic and efficient approach to identifying the optimal GRU model configuration for stock price prediction.

4. Experiments and Results

4.1. Evaluation Metrics

To evaluate the performance of our models, we utilized the following metrics:

1. **Mean Squared Error (MSE):** MSE measures the average squared difference between the predicted and actual values. A lower MSE indicates better performance, as it penalizes large errors more heavily.

2. **Root Mean Squared Error (RMSE):** RMSE is the square root of MSE and provides a more interpretable metric by returning the error in the same unit as the target variable. It helps in assessing how close the predicted values are to the actual ones.

3. **Mean Directional Accuracy (MDA):** MDA measures the percentage of times the predicted and actual values move in the same direction. A higher MDA signifies better directional forecasting.

These metrics comprehensively evaluate the models' ability to minimize errors and capture directional movements effectively.

We also analyzed the training and validation loss graphs to monitor convergence and potential overfitting during training. True versus predicted price graphs were generated to visualize the models' prediction accuracy over time.

Table 1. Comparison of Performance Metrics Across Models

Metric	Simple RNN	LSTM	GRU
MSE	0.000235	0.000149	0.000123
RMSE	0.015330	0.012221	0.011122
MDA	0.5045	0.4685	0.4639

4.2. Model Performance

4.2.1 Simple RNN Model Performance

Key insights from the model's performance include:

- **Accuracy of Prediction:** The **Simple RNN** achieved a normalized MSE of **0.000235**, RMSE of **0.01533**, and MDA of **0.5045**, as shown in Table 1.
- **Alignment with True Prices:** The *True vs Predicted Prices* graph (Figure 7) demonstrates the model's ability to capture the trends of stock prices accurately. Predicted values closely track true prices across most time steps.
- **Handling Fluctuations:** While the model effectively follows overall price trends, minor deviations are observed, particularly during periods of rapid price changes, indicating potential areas for further model refinement.

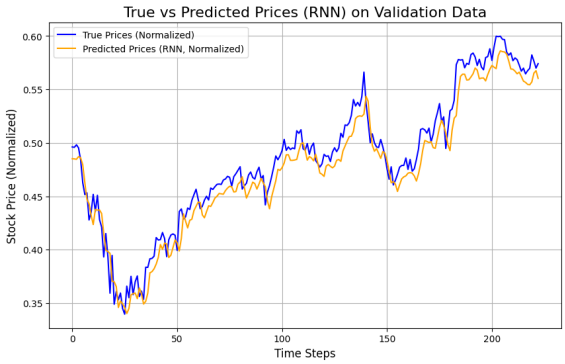


Figure 7. True vs Predicted Prices for Simple RNN on Validation Data.

4.2.2 LSTM Performance

The Long Short-Term Memory (LSTM) model demonstrated robust performance in predicting stock prices, as highlighted in Table 1 and the True vs. Predicted graph shown in Figure 8. Below are the key insights:

- The normalized Mean Squared Error (MSE) for the LSTM model was 0.000149, reflecting a high level of accuracy in capturing the underlying trends in stock prices.
- The Root Mean Squared Error (RMSE) stood at 0.01222, indicating a low deviation between the predicted and true prices.
- The Mean Directional Accuracy (MDA) was 46.85%, showcasing its moderate ability to predict the correct direction of stock price movements.

- The True vs. Predicted graph (Figure 8) exhibits strong alignment, with the predicted values closely following the actual stock price trends, especially during upward and downward movements.

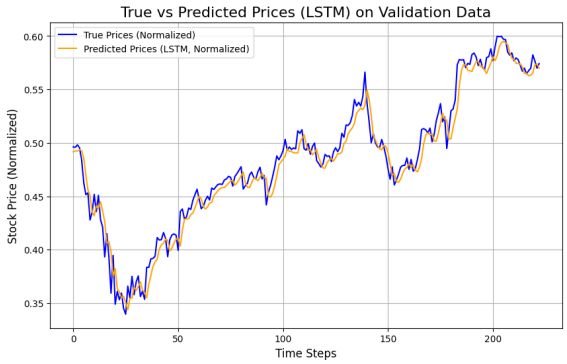


Figure 8. True vs Predicted Stock Prices (LSTM) on Validation Data

4.2.3 GRU Model Performance

The GRU model's performance metrics, as summarized in Table 1, show its efficacy in stock price prediction. The Normalized Mean Squared Error (MSE) is 0.0001237, and the Normalized Root Mean Squared Error (RMSE) is 0.01112. The Mean Directional Accuracy (MDA) metric is 0.46396, indicating the model's ability to predict the correct directional trend for almost half of the instances.

Figure 9 visualizes the true versus predicted stock prices, demonstrating the GRU model's capacity to closely follow the actual stock price trends. The model captures the overall dynamics and fluctuations, with minor deviations at certain intervals. This ability to maintain proximity to true values underscores the GRU model's robustness in temporal sequence predictions.

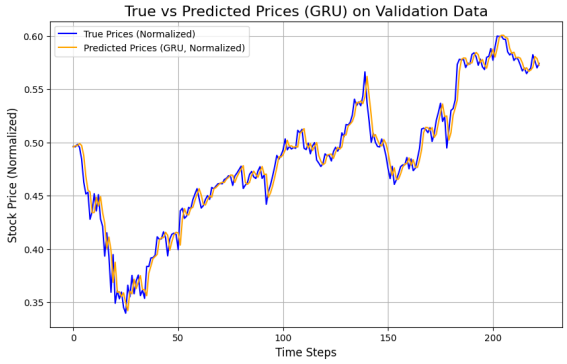


Figure 9. True vs Predicted Stock Prices for GRU Model on Validation Data

Key insights from the GRU model evaluation include:

- The GRU model achieves the lowest Normalized MSE among all tested models, signifying its strength in minimizing prediction errors.
- The Normalized RMSE further confirms the GRU's reliability, indicating its accurate approximation of stock price variations.
- Although the MDA of 0.46396 is slightly lower than ideal, the model effectively captures the underlying stock price trends, as evidenced by the comparison of true and predicted prices.
- Denormalized values of true and predicted stock prices, further highlight the GRU's high accuracy, with predictions consistently aligning closely to actual stock prices.

4.3. Comparison Analysis and Best Model Selection

To evaluate the performance of the models, weighted scores were calculated based on the normalized metrics (MSE, RMSE, and MDA) using predefined weights. The results of the performance scores for each model are summarized in Table 2.

Table 2. Model Performance Scores

Model	Performance Score
Simple RNN	2147.1772
LSTM	3372.0745
GRU	4068.9723

Key findings from the analysis:

- The GRU model achieved the highest performance score (4068.9723), demonstrating its superior ability to capture temporal dependencies and predict stock prices accurately.
- The LSTM model followed with a score of 3372.0745, reflecting its effectiveness in learning long-term dependencies but slightly underperforming compared to GRU.
- The Simple RNN model achieved a score of 2147.1772, indicating its limited capacity to handle complex sequential data compared to GRU and LSTM.

Based on these results, the GRU model was identified as the best-performing model, owing to its higher weighted score and closer alignment between true and predicted stock prices, as depicted in Figure 9.

4.4. Hyperparameter Tuning Outcome

The hyperparameter tuning process aimed to identify the optimal combination of hyperparameters to improve the performance of the GRU model. A random search was conducted over a predefined hyperparameter space, and ten hyperparameter combinations were evaluated based on their validation loss. The best-performing hyperparameters are highlighted.

Table 3. Best Hyperparameters for the Optimized GRU Model

Hyperparameter	Value
Units	64
Learning Rate	0.01
Number of Layers	1
Epochs	100
Dropout	0.1
Batch Size	64

With these hyperparameters, the GRU model achieved the lowest validation loss of **0.00014766**. This result confirms the effectiveness of hyperparameter optimization in improving model performance.

4.5. Test Set Evaluation with Optimized GRU Model

The optimized GRU model, derived from hyperparameter tuning, was used for evaluation on the test dataset. The results highlight the model's ability to generalize to unseen data while maintaining competitive performance metrics.

Table 4. Test Data Performance Metrics (Normalized)

Metric	Value
MSE	0.001766
RMSE	0.042030
MDA	0.486150

The performance of the optimized GRU model was assessed using the Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Directional Accuracy (MDA). These metrics quantify the model's predictive accuracy and ability to capture directional trends in stock prices. The normalized evaluation metrics for the test dataset are presented in Table 4.

To visually assess the model's performance, a comparison of the true and predicted stock prices on the test dataset is illustrated in Figure 10. The results demonstrate that the GRU model closely tracks the actual stock price trends with minor deviations, particularly in periods of high volatility.

The training and validation loss curves during the model's training phase are shown in Figure 11. The optimized GRU model exhibits stable convergence, with no signs of overfitting or underfitting.

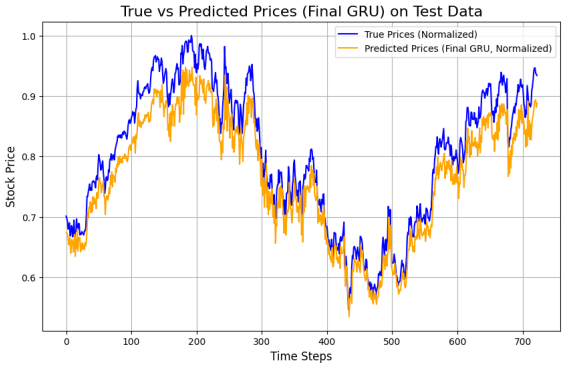


Figure 10. True vs Predicted Prices (GRU) on Test Data.

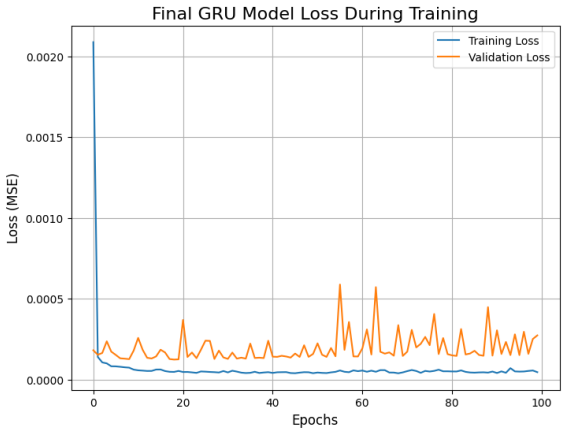


Figure 11. Final GRU Model Loss During Training.

Table 5. Comparison of True and Predicted Prices (Denormalized) on Test Data

Index	True Price	Predicted Price
0	106.15	102.14
1	105.60	102.00
2	104.80	101.59
3	102.99	100.86
4	103.29	99.13
5	104.50	100.12
6	101.32	100.95
7	101.59	97.08
8	103.82	99.07
9	103.54	100.43

To evaluate the accuracy in real-world terms, the denormalized true and predicted stock prices are compared in Table 5. The predicted prices align well with the actual values, showcasing the model's capability in replicating real stock trends.

5. Discussion

The evaluation of Simple RNN, LSTM, and GRU models on the Google stock price dataset highlights key insights into the performance and suitability of each architecture for time series prediction. Among these, the GRU emerged as the most effective model with the best performance metrics on both validation and test datasets. This is attributed to GRU's simplified gating mechanism, which effectively balances computational efficiency and predictive accuracy. The GRU model achieved a normalized Mean Squared Error (MSE) of 0.0001237 and a Root Mean Squared Error (RMSE) of 0.0111, underscoring its capacity to capture temporal dependencies and trends in stock prices.

In comparison, the LSTM model performed slightly worse, with a normalized MSE of 0.0001494 and RMSE of 0.0122. While LSTM's gating mechanism offers superior control over long-term dependencies, its increased complexity may lead to overfitting or difficulty in converging with limited data. The Simple RNN, as expected, lagged behind with an MSE of 0.0002350 and RMSE of 0.0153, highlighting its limitations in capturing long-term dependencies due to the lack of sophisticated gating mechanisms. These results demonstrate the significant advantages of modern recurrent architectures like GRU and LSTM over traditional RNNs in handling sequential data.

The findings emphasize the critical role of hyperparameter optimization and architectural design in achieving superior performance. The optimized GRU model, with its specific configuration of 64 units, a learning rate of 0.01, and a dropout rate of 0.1, demonstrated strong generalization and robustness in predicting stock prices. Additionally, the True vs. Predicted graphs showcased GRU's ability to closely follow the trend of actual prices, reducing the gap between observed and predicted values.

6. Conclusion

This study demonstrates the effectiveness of recurrent neural networks, particularly GRU, in predicting stock prices based on historical data. The GRU model outperformed both Simple RNN and LSTM, achieving the lowest error metrics and closely aligning predicted values with actual trends. This highlights the importance of selecting appropriate architectures and optimizing hyperparameters for time series forecasting tasks. Throughout the process, I gained valuable insights into the strengths and limitations of different recurrent architectures, as well as the critical role of preprocessing and hyperparameter tuning in enhancing model performance. The True vs. Predicted graphs further validated the models' ability to capture temporal dependencies.

For future work, exploring advanced techniques such as transfer learning, ensembling multiple models, or incorporating external market indicators could further improve prediction accuracy and robustness, making the approach more suitable for real-world applications.

Code

The codebase and data are made available on GitHub to facilitate reproducibility and further exploration. https://github.com/ruhinb/Stock-Price-Prediction-Using-Recurrent-Neural-Networks

References

[1] Anishnama. Understanding gated recurrent unit (GRU) in deep learning. 4

[2] Kyunghyun Cho, B van Merrienboer, Caglar Gulcehre, F Bougares, H Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. 1

[3] Thomas Fischer and Christopher Krauss. Deep learning with long short-term memory networks for financial market predictions. 270(2):654–669. 1

[4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep learning. 2

[5] Jigar Patel, Sahil Shah, Priyank Thakkar, and K Kotecha. Predicting stock market index using fusion of machine learning techniques. 42(4):2162–2172. 2

[6] Sushmita Poudel. Recurrent neural network (RNN) architecture explained. 3

[7] Harya Widiputra, Adele Mailangkay, and Elliana Gautama. Multivariate cnn-lstm model for multiple parallel financial time-series prediction. *Complexity*, 2021, 10 2021. 4